

Abstraction

يعني ايه abstraction بالمختصر المفيد هو عبارة كلاس او ميثود مش كامل أي مش معمله implementation ومطلوب من الكلاسات التانيه نكملة

طيب تعرف ازاي ان الكلاس ده abstract وهو الا بيكتب قبل ال access modifier او بعدها كلمه abstract

مثال:

```
abstract class Animal {  
    abstract void makeSound(); // مقيش body  
}
```

بص هنا كاتب قبل الكلاس كلمه abstract وكمان قبل ال ميثود abstract

طيب ايه هي أنواع ال abstract

abstract method-1

وهي عبارة عن داله مفيهاش body اي مش معملها implementation

وكمان لازم علشان تكون abstract تكون داخل كلاس abstract

طيب لو جه كلاس يورث كلاس ال الداله دي فيه لازم يعمل override للدله دي

```
abstract class Animal {  
    abstract void makeSound(); // مقيش body  
}
```

دي شكل الميثود جوه كلاس الاتنين معمول لهم abstract هتقولي طب ازاي ال override انت قلت عليها دي اقولك استنا على رزق وقولك عليها تحت

دلوقت هنتكلم عن النوع الثاني من ال abstract وهو

abstract class-2

طيب ايه شروطه

اول حاجه انك متنفعش تعمل منه object

وممكن يحتوي على ال abstract method او non-abstract methods وكممكن يكون فيه constructor

```
abstract class Animal {
    String name;

    Animal(String name) {
        this.name = name;
    }

    abstract void makeSound(); // لازم يتكتب في الكلاسات اللي ترث

    void sleep() {
        System.out.println(name + " is sleeping");
    }
}
```

هنا الكلاس abstract animal وجوه ميثود (makeSound) abstract وكممكن جوه non-abstract method (sleep) وفيه constructor

طيب خلاصة الكلام الفوق

✦ أهم القواعد:

الشرح

القاعدة

Animal a = new Animal(); ✗

لا يمكن إنشاء object من abstract class

وإلا الكود مش هيشغل

abstract method لازم تكون في abstract class

الكلاس اللي يرث abstract لازم يطبق كل الدوال المجردة

أو يبقى هو كمان abstract

مممكن تكتب دوال عادية وجواها كود في abstract class

عادي جدًا

```

abstract class Animal {
    String name;

    Animal(String name) {
        this.name = name;
    }

    abstract void makeSound(); // دالة مجردة

    void sleep() {
        System.out.println(name + " is sleeping");
    }
}

class Dog extends Animal {
    Dog(String name) {
        super(name);
    }

    @Override
    void makeSound() {
        System.out.println(name + " says: Woof!");
    }
}

```

ده كود شامل كل الكلام ال احنا قولناه

هو الكود عبارته عن abstract class(animal)

Constructor وabstract class(makesound) وnon-abstract class(sleep)

واهم حتة ال قولت اشرحها تحت وهي عندما كلاس يورث من كلاس abstract هنا dog ووارث من animal ببص بقى على ميثود ال كانت ابسترا د في كلاس الاب هتلاقيه هنا هو عامله override وكم ان عملها implementation

طب هل في علاقه بين العلاقة بين Polymorphism وAbstract ؟

اقولك abstract class هو واحد من أهم أدوات تنفيذ polymorphism

طب يعني ايه أصلاً polymorphism ان شاء الله أتكلّم عنه بالتفصيل في report تاني بس اقولك عليه نبذه علشان تفهم الجزائيه ال عليها الدور

Polymorphism

يعني: نفس الدالة أو نفس الواجهة تشتغل بطرق مختلفة على كائنات مختلفة.

مثلاً:

- كل الكائنات عندها دالة makeSound()، لكن الكلب بيهو هو، والقطعة بتموء.

- بننده على نفس الدالة، بس كل كائن بينفذها بطريقته.

هتقولي برضو مش فاهم اقولك ان شاء الله هتفهم من الكود بس

```
abstract class Animal {  
    abstract void makeSound();  
}
```

```
class Dog extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Woof!");  
    }  
}  
  
class Cat extends Animal {  
    @Override  
    void makeSound() {  
        System.out.println("Meow!");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        Animal a1 = new Dog(); // object من Dog لكن نوعه Animal  
        Animal a2 = new Cat();  
  
        a1.makeSound(); // Woof!  
        a2.makeSound(); // Meow!  
    }  
}
```

بص يعم هنا هتكلم عن جزئيه polymorphism وabstract بس هنا احنا عرفنا كلاس abstract animal ووجوه
method abstract makesound بص بقى هو ورثها للكلاس ال dog و cat الشغل كله في ال main هنا كلنا
عرفين ال مينفعش نعمل object من abstract class هتقولي هو عامل اهو اقولك لا انا لو لاحظته جه قالك بعد
علامه اليساوي من النوع dog يعني اول ما الكومبيلر يرن هيروح يستدعي الداله ال في ال dog مش ال في animal
وكذلك في ال cat

طبيب يجي واحد يقولي انت بتتكلم ده كله عن ال abstract طب انا اعمله ليه او استخدمه ليه مانا شغال زي الفل من غيره اقولك انا بقي

لانه بيدي الكود بتاعك تنظيم قوي جدا وخصوصا لما المشروع يكون كبير

مثال :

لو انت مثلا ليدر وعايز تفرض قاعده او شكل مشترك بين التيم بتاعك

زي مثلا

```
abstract class Animal {  
    abstract void makeSound();  
}
```

أي حد من التيم ايعمل توريث من الكلاس ال animal مجبر ميكملش الكود غير لما يعمل implementation للميثود ال اسمها makesound ده مثال بسيط

تاني حاجه تقليل التكرار (Code Reuse)

تقدر تكتب كود مشترك لكل الكائنات في abstract class، وتخلي كل نوع يكتب بس التفاصيل اللي بتختلف.

3-تبني نظام مرن (Flexible System)

بتخلي الكود مرن، يعني لو بعدين حبيت تضيف نوع جديد من الكائنات، مش محتاج تغيّر الكود القديم، بس تعمل كلاس جديد يرث من ال abstract class ويطبق المطلوب.

الخلاصه

بدون abstract

حرية كاملة لكن فوضى محتملة

صعب توسّع الكود بأمان

تكرار كبير في الكود

باستخدام abstract

تنظيم + إجبار الكلاسات تمشي على شكل معين

سهل تضيف وتغير من غير ما تخرب القديم

إعادة استخدام للكود المشترك